

Nom :

Prénom :

Groupe :

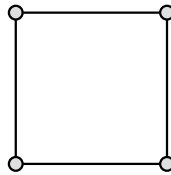
Université Clermont Auvergne
BUT Info 1A
R207 - Graphes
2023-2024

Devoir surveillé 1
Partie 1
8,5 points

Cécile Bertrand
Florent Foucaud
Anis Fradi
Malika More
Carine Simon

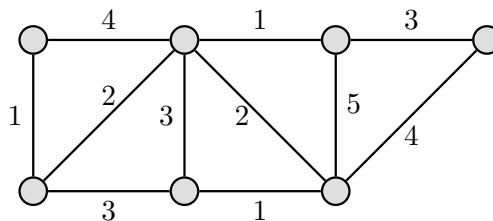
PARTIE A – Graphes et arbres (5 points)

1. (1 pt) Pour chaque assertion ci-dessous, écrivez si elle est vraie ou fausse en **justifiant votre réponse** (par exemple, vous pouvez donner un exemple pour lequel une assertion est fausse).
- (a) Il existe 4 arbres couvrants différents du graphe ci-dessous :



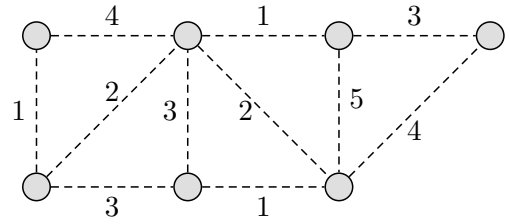
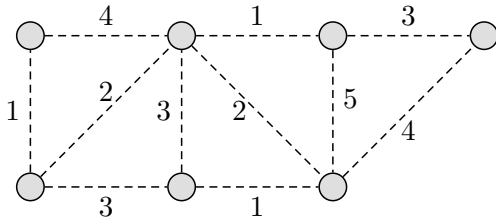
- (b) Un graphe a n sommets et $n - 1$ arêtes est toujours un arbre (avec n entier naturel non nul).

2. (1,5 pt) On considère le graphe G valué non orienté suivant :



- (a) Combien faut-il enlever d'arêtes à ce graphe pour le transformer en arbre ? Justifiez.

- (b) Représentez sur les canevas ci-dessous 2 arbres couvrants différents du graphe G (pas forcément de poids minimum) et précisez leur poids.

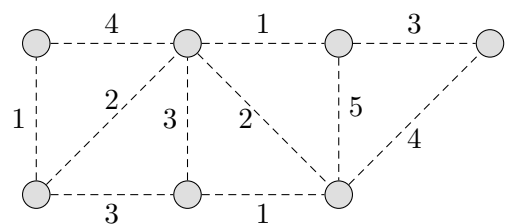
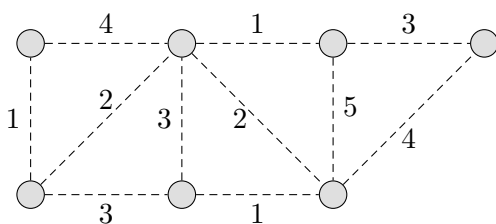
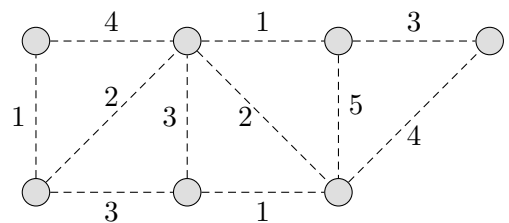
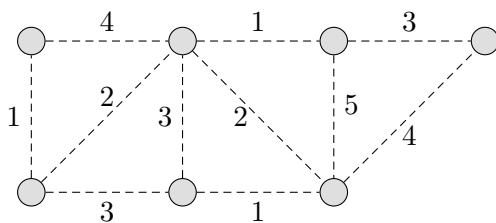
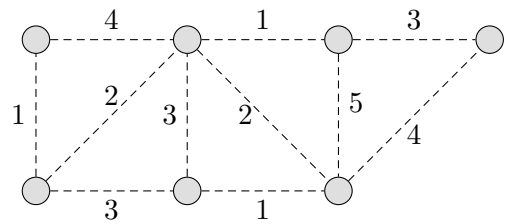
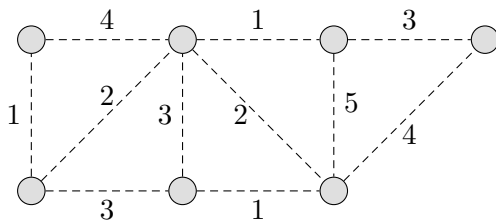
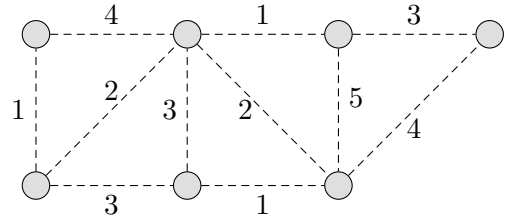
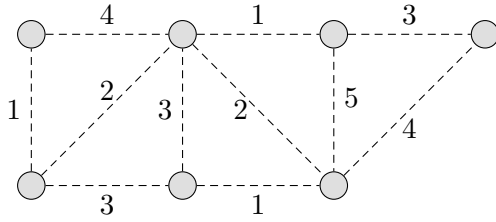
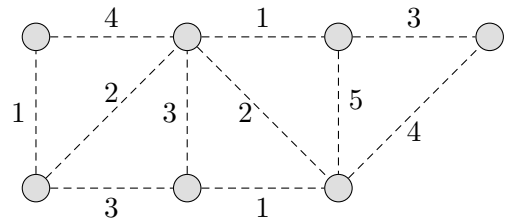
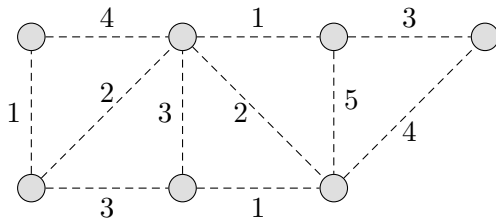


poids de l'arbre couvrant obtenu :

poids de l'arbre couvrant obtenu :

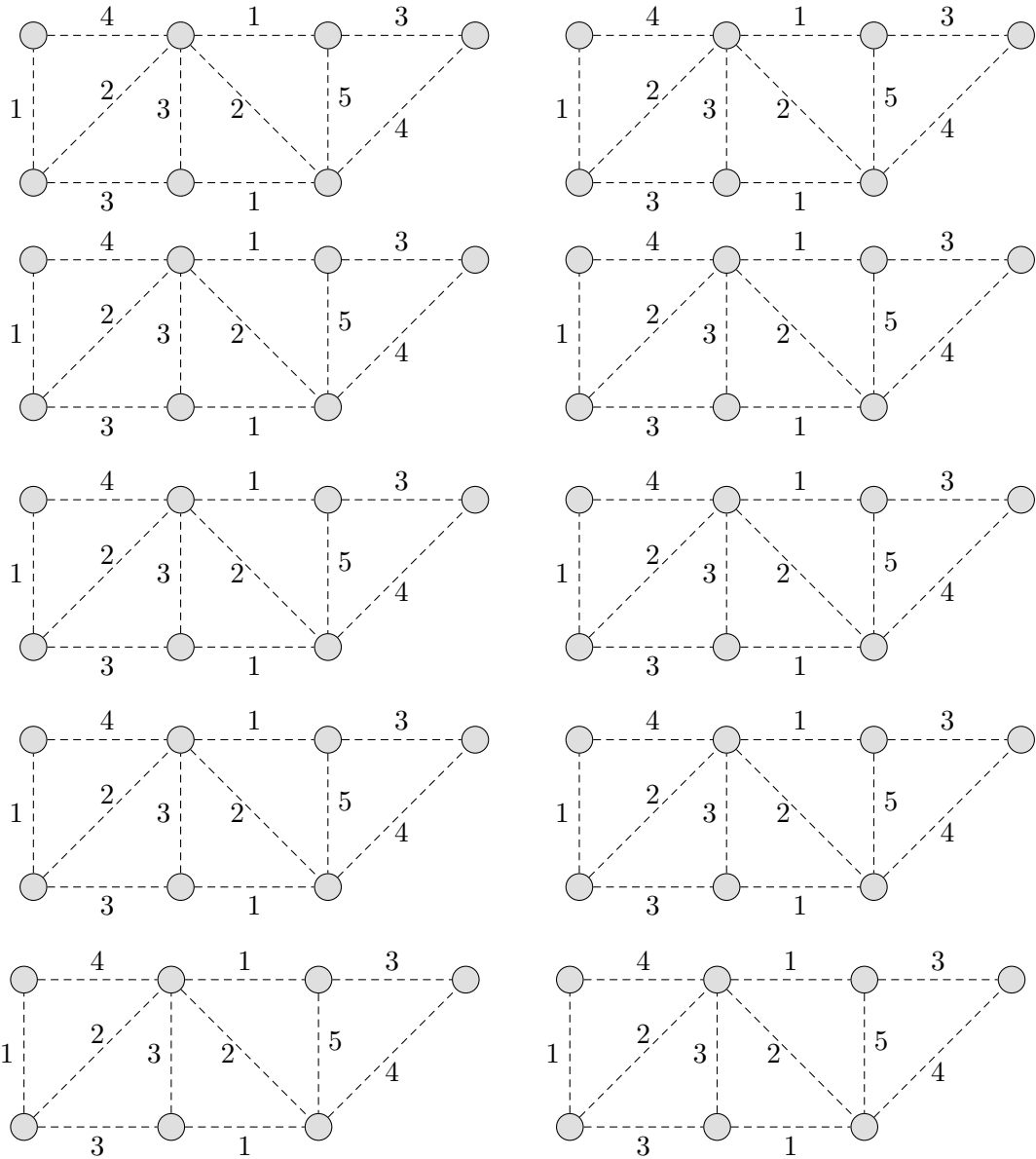
3. (2,5 pts) On désire calculer un arbre couvrant de poids minimum pour le graphe G .

- (a) Appliquez l'algorithme de PRIM en commençant par le sommet en haut à gauche (détaillez bien chaque étape en utilisant le canevas ci-dessous) puis précisez le poids minimal.



poids minimal :

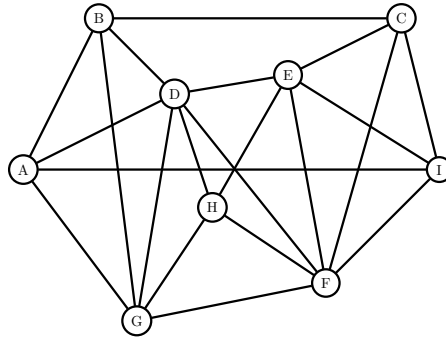
- (b) Appliquez l'algorithme de KRUSKAL (détaillez bien chaque étape en utilisant le canevas ci-dessous) puis précisez le poids minimal.



poids minimal :

PARTIE B – Graphes : vocabulaire et matrices (3,5 points)

1. (1,5 pt) On considère le graphe ci-dessous :



(a) Le graphe est-il connexe ? Justifiez.

(b) Compléter le tableau ci-dessous avec les degrés de chaque sommet. Déduisez-en, en justifiant, le nombre d'arêtes du graphe.

sommets	A	B	C	D	E	F	G	H	I
degrés									

(c) Le graphe est-il complet ? Si non, combien d'arêtes faut-il ajouter pour qu'il soit complet ? Justifiez.

2. (1 pt) On considère ci-dessous la matrice d'adjacence d'un graphe. Sans dessiner le graphe, répondez aux questions suivantes en justifiant les réponses uniquement à l'aide de la matrice.

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \end{pmatrix}$$

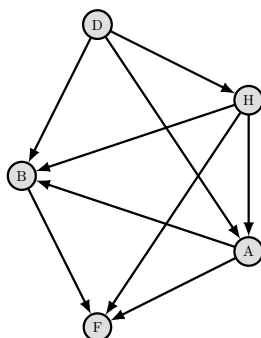
(a) Le graphe est-il orienté ?

(b) Le graphe contient-il des boucles ?

(c) Quel est le degré du 1er sommet ?

(d) Le graphe est-il complet ?

3. (1 pt) Un parcours sportif est composé d'un banc pour abdominaux, de haies et d'anneaux. Le graphe orienté ci-dessous indique les différents parcours conseillés partant de D et terminant à F. Les sommets sont D (départ), B (banc pour abdominaux), H (haies), A (anneaux) et F (fin du parcours). Les arcs représentent les différents sentiers reliant les sommets.

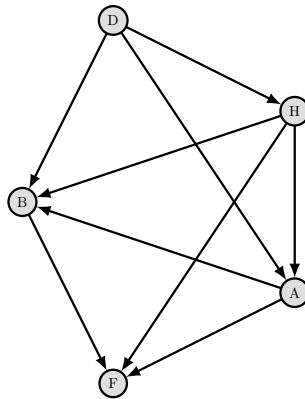


(a) Donnez la matrice d'adjacence M de ce graphe.

- (b) Le responsable souhaite ajouter une barre de traction notée T . De nouveaux sentiers sont construits et de nouveaux parcours sont possibles. La matrice d'adjacence N associée au graphe représentant les nouveaux parcours dans lequel les sommets sont classés par ordre alphabétique, est :

$$N = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Complétez le graphe ci-dessous en ajoutant les arcs nécessaires pour que le graphe orienté obtenu corresponde à la matrice N .



(T)

Nom :

Prénom :

Groupe :

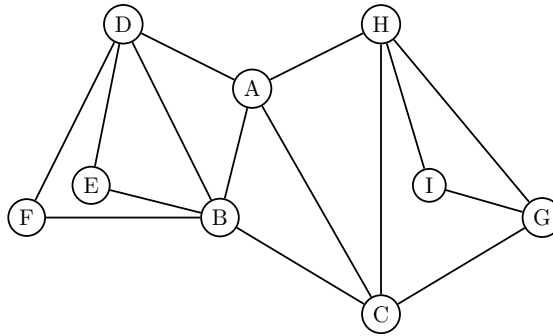
Université Clermont Auvergne
BUT Info 1A
R207 - Graphes
2023-2024

Devoir surveillé 1
Partie 2 : Parcours et distances

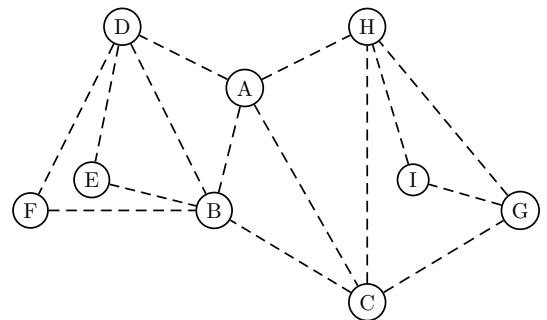
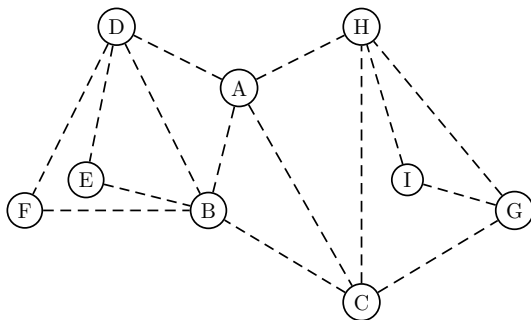
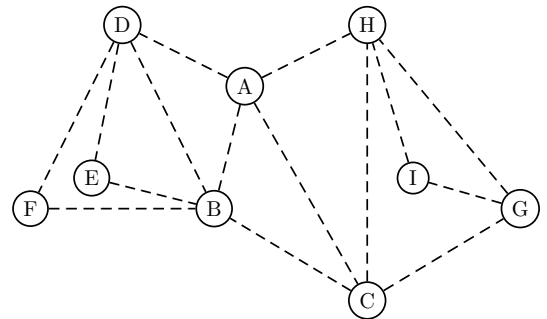
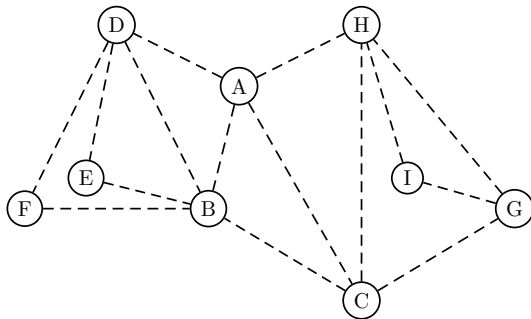
8,5 points

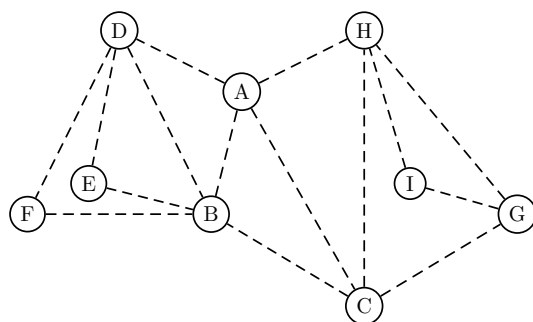
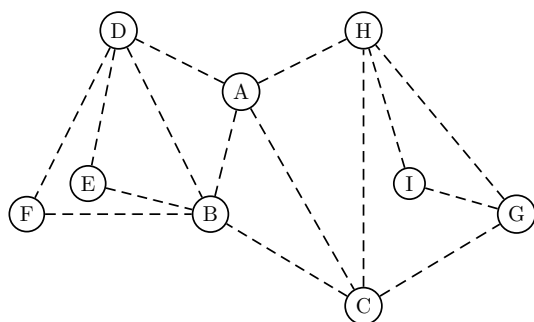
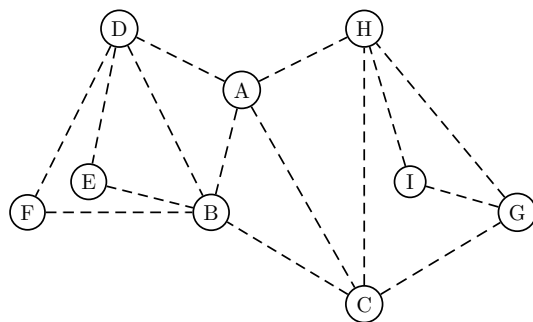
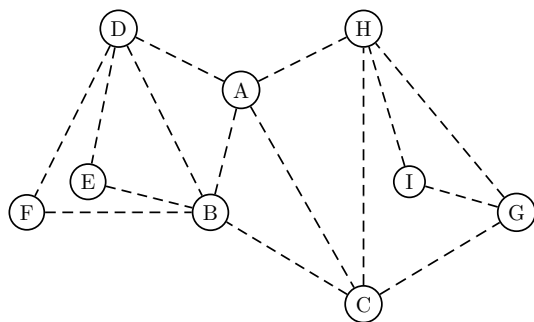
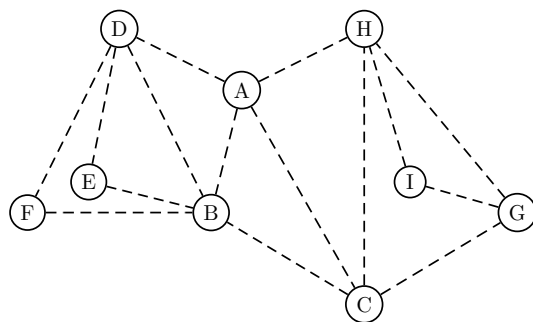
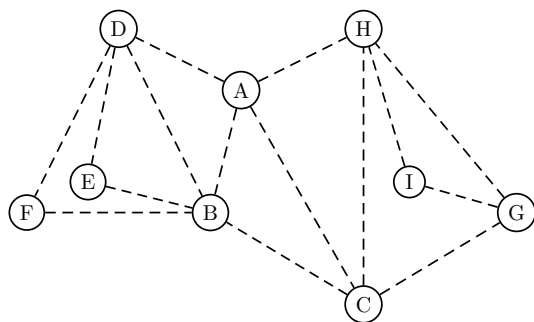
Cécile Bertrand
Florent Foucaud
Anis Fradi
Malika More
Carine Simon

1. On considère le graphe $\mathcal{G}$ ci-dessous.

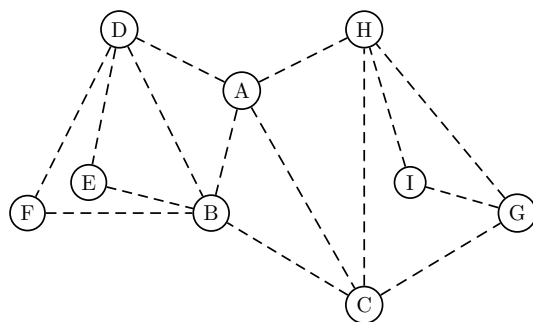
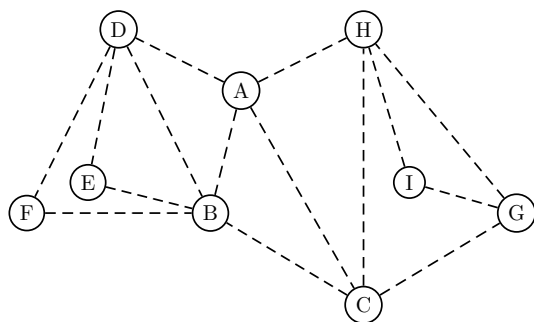


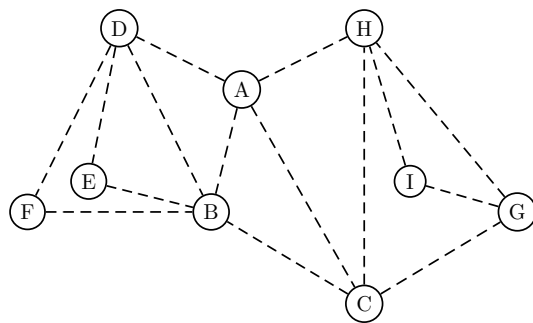
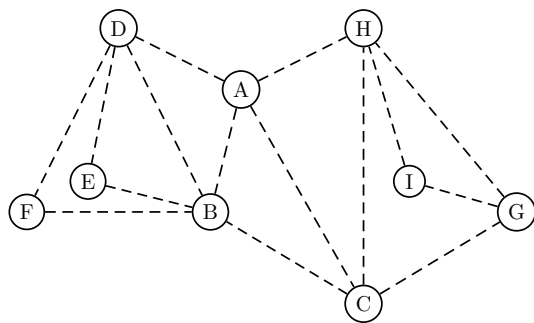
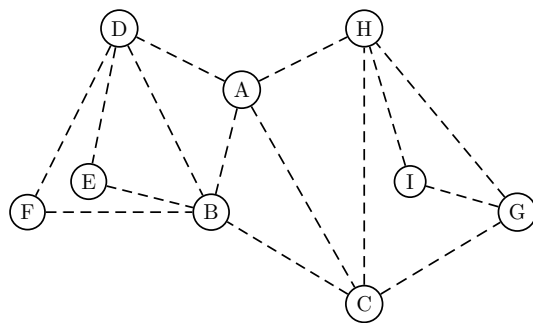
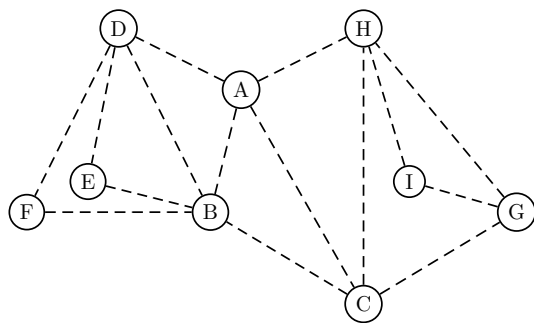
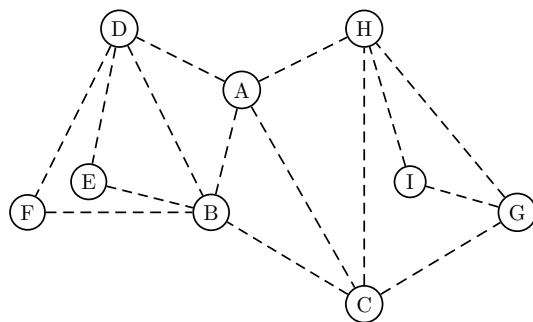
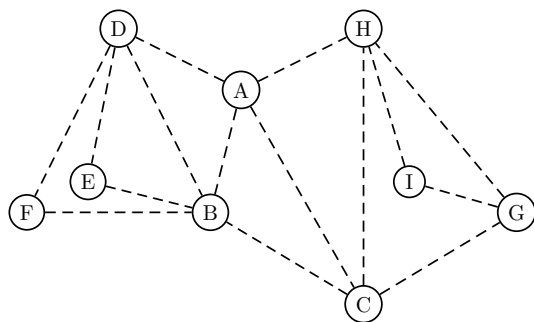
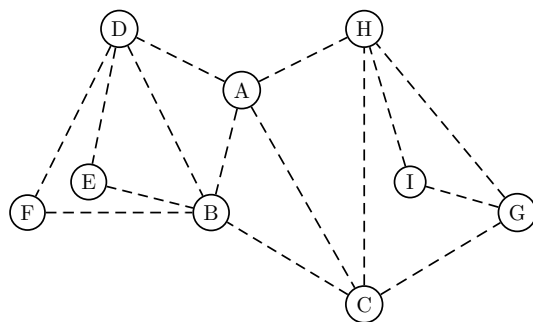
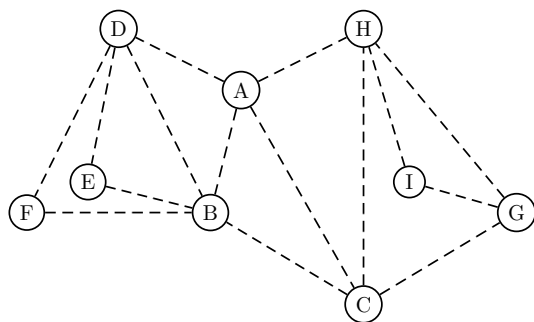
- (a) (1 points **) Effectuez un parcours en largeur de $\mathcal{G}$ à partir du sommet D . Détaillez chaque étape en utilisant le canevas ci-dessous, et indiquez sous chaque schéma l'évolution de la structure de données. On conviendra de toujours examiner les voisins d'un sommet dans l'ordre alphabétique.



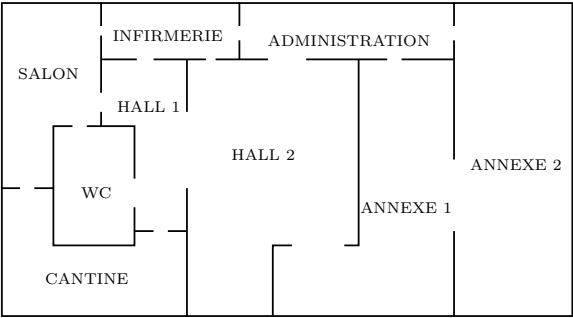


(b) (1 points **) Effectuez un parcours en profondeur de $\mathcal{G}$ à partir du sommet D . Détaillez chaque étape en utilisant le canevas ci-dessous, et indiquez sous chaque schéma l'évolution de la structure de données. On conviendra de toujours examiner les voisins d'un sommet dans l'ordre alphabétique.





2. On donne ci-dessous le plan simplifié d'un bâtiment.



(a) (0,5 points **) Complétez le tableau suivant, et expliquez comment le graphe $\mathcal{G}$ de la question 1 peut modéliser la situation.

Sommet de $\mathcal{G}$	A	B	C	D	E	F	G	H	I
Lieu correspondant dans le bâtiment									

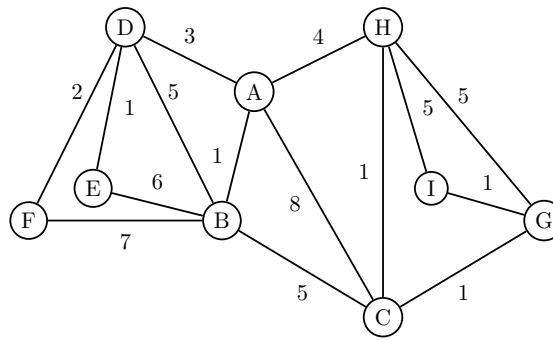
(b) Au cours d'un exercice, les pompiers utilisent un fumigène pour simuler un départ d'incendie dans le salon du bâtiment. On considère que chaque minute, la fumée envahit toutes les salles qui communiquent par une porte avec une salle enfumée. En utilisant la question 1, répondez aux questions suivantes.

i. (0,5 points *) Lequel des deux parcours de graphes est le plus proche de cette situation ? Justifiez votre réponse.

ii. (0,5 points *) Quel est l'ensemble des salles qui seront enfumées à un moment ou à un autre ? Justifiez votre réponse.

iii. (0,5 points *) Décrivez la propagation de la fumée au cours du temps. Justifiez votre réponse.

On considère le graphe pondéré ci-dessous.



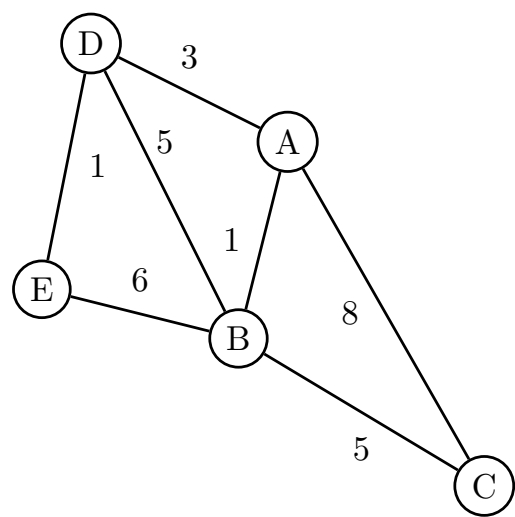
(a) (1,5 points **) En appliquant l'algorithme de Dijkstra, calculez la distance entre le sommet D et chaque autre sommet du graphe en terminant de remplir le tableau ci-dessous.

A	B	C	D	E	F	G	H	I
$+\infty$	$+\infty$	$+\infty$	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
3	5	$+\infty$	X	1	2	$+\infty$	$+\infty$	$+\infty$
3	5	$+\infty$	X	X	2	$+\infty$	$+\infty$	$+\infty$
3	5	$+\infty$	X	X	X	$+\infty$	$+\infty$	$+\infty$

(b) (1 points *) Indiquez le chemin le plus court entre D et I , en montrant sur le tableau comment vous l'avez trouvé.

4. Dans cette question, on s'intéresse à l'exécution de l'algorithme de Floyd-Warshall sur le graphe orienté valué $\mathcal{H}$ ci-dessous.

(a) (0,5 points *) Remplissez la matrice initiale ci-dessous.



	A	B	C	D	E
A					
B					
C					
D					
E					

(b) (1 points **) Exécutez les trois itérations suivantes sur les matrices ci-dessous, en considérant les sommets dans l'ordre alphabétique. **On ne vous demande pas de terminer l'exécution de l'algorithme.**

	A	B	C	D	E
A					
B					
C					
D					
E					

	A	B	C	D	E
A					
B					
C					
D					
E					

	A	B	C	D	E
A					
B					
C					
D					
E					

(c) (0,5 points *) Si on programme l'algorithme, pourquoi peut-on éviter de faire une copie de la matrice à chaque itération ? Pourquoi faire une telle copie peut quand même être utile ?

Nom :

Prénom :

Groupe :

Université Clermont Auvergne
BUT Info 1A
R207 - Graphes
2023-2024

Devoir surveillé 1
Partie 3 : QCM Python
4 points

Cécile Bertrand
Florent Foucaud
Anis Fradi
Malika More
Carine Simon

Pour chacune des questions suivantes, veuillez cocher l'**unique bonne réponse**. Chaque question correcte rapporte 0,5 point. Une mauvaise réponse coûte 0,25 points négatifs (seulement au sein de cette partie). Une absence de réponse n'est pas pénalisée.

1. Laquelle des instructions suivantes **ne renvoie pas** le nombre d'arêtes de G ?

- | | |
|--------------------------------------|-------------------------------------|
| (a) <code>G.number_of_edges()</code> | (c) <code>G.edges</code> |
| (b) <code>G.size()</code> | (d) <code>len(list(G.edges))</code> |

2. Laquelle des instructions suivantes **ne vaut pas** `True` si le sommet s de G a au moins un sommet voisin ?

- | | |
|---|---|
| (a) <code>len(G.nodes(s)) > 0</code> | (c) <code>len(G.edges(s)) > 0</code> |
| (b) <code>len(G.neighbors(s)) > 0</code> | (d) <code>G.degree(s) > 0</code> |

3. Laquelle des instructions suivantes **ne permet pas** de parcourir les sommets voisins du sommet s ?

- | | |
|--|--|
| (a) <code>for v in G.nodes:</code>
<code>if s in G.edges(v):</code> | (c) <code>for v in G.nodes:</code>
<code>if G.has_edge(s,v):</code> |
| (b) <code>for v in G.neighbors(s):</code> | (d) <code>for v in G.edges(s):</code> |

4. Que calcule la fonction ci-dessous ?

```
def mystere(G):  
    res = len(G.nodes)  
    for s in G.nodes:  
        d = G.degree(s)  
        if d < res:  
            res = d  
    return res
```

- | | |
|-----------------------------|-------------------------------|
| (a) Le degré de s | (c) Le degré maximum de G |
| (b) Le degré minimum de G | (d) Le nombre d'arêtes de G |

5. Comment compléter le code ci-dessous pour que G devienne un tournoi transitif à n sommets ?

```
G = nx.DiGraph(n)  
for i in range(0,n):  
    G.add_node(i)  
for i in range(0,n):  
    ### LIGNE MANQUANTE ###  
    G.add_edge(i,j)
```

- | | |
|---------------------------------------|---|
| (a) <code>for j in range(0,n):</code> | (c) <code>for j in range(i,n-1):</code> |
| (b) <code>for j in range(0,i):</code> | (d) <code>for j in range(i+1,n):</code> |

6. Quelle suite d'instructions permet de savoir si G contient un sommet de degré 3 ?

- (a)

```
flag = False
for v in G.nodes:
    flag = len(list(G.neighbors(v))) == 3
```
- (b)

```
for v in G.nodes :
    if len(list(G.neighbors(v))) == 3:
        flag = False
```
- (c)

```
flag = True
for v in G.nodes:
    flag = not len(list(G.neighbors(v))) == 3
```
- (d)

```
flag = False
for v in G.nodes :
    if len(list(G.neighbors(v))) == 3:
        flag = True
```

7. On veut coder la fonction `aretes_traversantes(G,S)` utilisée dans l'algorithme de Prim. Comment compléter la ligne manquante ?

```
#calcule l'ensemble des arêtes entre S et le reste du graphe
def aretes_traversantes(G,S):
    res = []
    for e in G.edges:
        ### LIGNE MANQUANTE ###
        res.append(e)
    return res
```

- (a) `if (e[0] in S and e[1] in S) or (e[0] not in S and e[1] not in S):`
- (b) `if e[0] in S and not e[1] in S:`
- (c) `if (e[0] in S and not e[1] in S) or (e[1] in S and not e[0] in S):`
- (d) `if e[0] in S or e[1] in S:`

8. Voici le code du parcours en profondeur. Comment compléter la ligne manquante ?

```
def parcours_profondeur(G,s):
    visites = []
    P = []
    visites.append(s)
    ajouter_pile(P,s)
    while(not est_pile_vide(P)):
        v = valeur_sommet_pile(P)
        trouve = False
        for u in G.neighbors(v):
            ### LIGNE MANQUANTE ###
            visites.append(u)
            ajouter_pile(P,u)
            trouve = True
            break
        if not trouve :
            enlever_sommet_pile(P)
    return visites
```

- (a) `if u not in P:`
- (b) `if u not in visites:`
- (c) `if v not in P:`
- (d) `if v not in visites:`